

RAPPORT TECHNIQUE

Moteur Morphologique Arabe

Projet : Mini-projet Algorithmique

Année : 2025-2026

Responsables : Narjes Ben Hariz, Sahbi Bahrour

Réalisatrices : Arij Sebai, Hadil Hasni, Balkis Hanafi (1ING3)

1. STRUCTURES DE DONNÉES UTILISÉES

1.1 Arbre AVL (Stockage des Racines)

Un **arbre AVL auto-équilibré** stocke les racines trilitères arabes. Chaque nœud contient une racine (ex: "كتب"), deux pointeurs enfants, une hauteur pour l'équilibrage, un cache de dérivés générés, et une map de fréquences d'utilisation. L'équilibrage automatique (différence hauteur ≤ 1) garantit des opérations en $O(\log n)$ même dans le pire cas. Le parcours infixe assure un affichage alphabétique des racines.

Justification : Pour 21 racines (dont 5 verbes irréguliers), l'AVL nécessite maximum **5 comparaisons** ($\log_2(21) \approx 4.4$) contre 21 dans le pire cas d'un arbre binaire simple. L'ordre lexicographique est requis pour l'affichage trié.

1.2 Table de Hachage (Stockage des Schèmes)

Une **table de hachage implémentée manuellement** avec chaînage gère les schèmes morphologiques. Chaque bucket contient une liste chaînée de nœuds (clé, valeur, suivant). La fonction de hachage **djb2** ($\text{hash} = \text{hash} * 33 + c$) offre une distribution uniforme sur les caractères UTF-8 arabes. Le redimensionnement automatique.

Caractéristiques : Capacité initiale 32 buckets, 11 schèmes stockés.

Justification : Les schèmes sont accédés à chaque génération de dérivé (opération la plus fréquente). La table de hachage offre $O(1)$ contre $O(\log 11) \approx 4$ comparaisons.

1.3 Listes Associées (Maps C++ STL)

Index inversé: `map<string, pair<string, string>> mot_to_racine_scheme` pré-calculé au démarrage. Pour chaque racine $\times$ schème, génère le dérivé et l'indexe par sa forme simplifiée (sans diacritiques). Contient ~ 231 entrées pour 21 racines $\times$ 11 schèmes, occupant ~ 10 KB.

Map de verbes irréguliers: `map<string, map<string, string>> verbes_irreguliers` stocke les formes spéciales de 5 verbes (مَدَّ، وَقَفَ، دَعَى، قَرَأَ، قَالَ) avec leurs 11 formes prédéfinies qui violent les règles morphologiques standards.

Cache de simplification: `unordered_map<string, string> cache_simplification` évite le recalcul répété de la suppression des 9 diacritiques arabes.

2. ALGORITHMES DE GÉNÉRATION ET VALIDATION MORPHOLOGIQUE

2.1 Génération Morphologique

Principe : La morphologie arabe repose sur l'insertion de 3 consonnes radicales dans un patron abstrait (schème) utilisant ل, ع, ف.

Algorithme détaillé :

- 1. **Extraction UTF-8 :** Parser manuel détectant les débuts de caractères multi-octets "ك", "ت" → "كتب", "تت"]
- 2. **Application avec marqueurs temporaires :** Remplace ف → \x01, ع → \x02, ل → \x03 pour éviter collisions si la racine contient ces lettres. Puis substitue marqueurs par consonnes radicales.
- 3. **Règles phonétiques :** Applique assimilations arabes (آ → إ, double hamza, etc.)

Complexité : $O(k \times m)$ où k = nombre de schèmes (11), m = longueur mot (~5 caractères). Total ~55 opérations par racine.

2.2 Validation Morphologique

Approche optimisée (implémentée) :

```
verifier_mot("كاتب"):
  1. Simplifier avec cache :  $O(1) \rightarrow$  "كاتب"
  2. Lookup index inversé :  $O(1) \rightarrow$  {"فاعل", "كتب"}
  3. Incrémenter fréquence dans AVL :  $O(\log n)$ 
  Total :  $O(1)$  en moyenne
```

2.3 Construction de l'Index Inversé

Algorithme (fonction `construire_index` appelée au démarrage) :

- 1. Extraire toutes les racines par parcours AVL : $O(n)$
- 2. Pour chaque racine (21×) : générer dérivés (11×) et indexer (1×)
- 3. Complexité totale : $O(n \times k) = O(231)$ une seule fois

Pré-calcul intelligent : Coût initial 22ms amorti sur milliers de validations futures à 1ms chacune.

3. CHOIX ET JUSTIFICATION DES ALGORITHMES

3.1 Comparaison des alternatives

Composant	Solution	Alternative	Justification
Racines	AVL	ABR simple	$O(\log n)$ garanti vs $O(n)$ pire cas; affichage alphabétique requis
Racines	AVL	HashSet	AVL offre ordre + $\log n$, HashSet perd ordre

Composant	Solution	Alternative	Justification
Schémes	HashTable	Vector	$O(1)$ vs $O(n)=11$ pour recherche fréquente
Schémes	HashTable	std::map	$O(1)$ vs $O(\log 11)\approx 4$; apprentissage implémentation
Validation	Index inversé	Génération itérative	$O(1)$ vs $O(990)$; 10KB mémoire acceptable
Hash	djb2	FNV-1a / Murmur	Simplicité + distribution excellente UTF-8

4. ANALYSE DE LA COMPLEXITÉ ALGORITHMIQUE

4.1 Complexités temporelles

Opération	Structure	Complexité	Justification
Insertion racine	AVL	$O(\log n)$	~5 comparaisons pour 21 racines
Recherche racine	AVL	$O(\log n)$	Équilibrage garanti
Recherche schème	HashTable	$O(1)$	Facteur charge 0.34, pas de collision
Génération dérivés	Morphologie	$O(k \times m)$	11 schèmes $\times$ 5 caractères $\approx$ 55 ops
Validation (optimisée)	Index inversé	$O(1)$	Lookup direct dans map
Verbes irréguliers	Map lookup	$O(1)$	5 verbes avec formes prédéfinies
Construction index	Pré-calcul	$O(n \times k)$	231 entrées générées au démarrage

5. PRINCIPALES DIFFICULTÉS RENCONTRÉES

5.1 Encodage UTF-8 des Caractères Arabes

Problème : Les caractères arabes (3 octets UTF-8) corrompus par indexation naïve `string[i]` (1 octet).
Solution : Parser UTF-8 manuel détectant débuts multi-octets (`(c & 0xF0) == 0xE0` $\rightarrow$ 3 octets) pour extraction correcte.

5.2 Performance de Validation

Problème : Approche naïve = 180ms/mot (21 racines $\times$ 11 schèmes $\times$ génération itérative).
Solution : Index inversé pré-calculé au démarrage $\rightarrow$ lookup $O(1)$ direct $\rightarrow$ < 1ms/validation (gain **180×**).

CONCLUSION

Ce moteur morphologique démontre l'efficacité des structures optimisées (AVL, HashTable, index inversé) pour l'analyse morphologique arabe. Les choix algorithmiques (validation $O(1)$, parsing UTF-8 manuel) assurent performance (gain 180×) et gestion correcte des 21 racines (dont 5 verbes irréguliers) avec 11 schèmes. L'architecture modulaire permet l'évolutivité vers des datasets plus importants.